

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 07 | Course: PARS Lab | Framework: Python 3.14 / Pure NumPy | Repository: [dSAxmonis/pars-lab](#)

Practical Name

Implementation of Linear Regression from Scratch Using Gradient Descent and Cost Curve Analysis in Python

AIM

To implement Linear Regression in Python from scratch without using any inbuilt machine learning library functions (such as `scikit-learn`), optimize parameters (weight w and bias b) via Gradient Descent for a real-world predictive application (Real Estate Price Prediction), and plot the Gradient Descent convergence cost curve alongside the fitted regression line.

THEORY

1. Hypothesis & Model Representation

For a scalar explanatory variable $x^{(i)}$, the linear regression hypothesis is represented as:

$$\hat{y}^{(i)} = w \times x^{(i)} + b$$

2. Cost Function (Mean Squared Error)

$$J(w, b) = (1 / 2m) \times \sum [(\hat{y}^{(i)} - y^{(i)})^2]$$

3. Analytical Gradients (Partial Derivatives)

$$\begin{aligned} \partial J / \partial w &= (1 / m) \times \sum [(\hat{y}^{(i)} - y^{(i)}) \times x^{(i)}] \\ \partial J / \partial b &= (1 / m) \times \sum [(\hat{y}^{(i)} - y^{(i)})] \end{aligned}$$

4. Simultaneous Parameter Update Rule

$$\begin{aligned} w &:= w - \alpha \times (\partial J / \partial w) \\ b &:= b - \alpha \times (\partial J / \partial b) \end{aligned}$$

STEPS & EXECUTION

Step 1: Scratch Implementation Framework

Formulated vectorized regression arithmetic strictly using fundamental NumPy operations without inbuilt estimators.

Step 2: Gradient Descent Optimization Loop

Executed gradient descent optimization tracking Mean Squared Error decay at each epoch.

Step 3: Web Application Deployment

Launched server at <http://127.0.0.1:5009> providing interactive hyperparameter tuning and visualization plots.

OUTPUT / ATTACHED SCREENSHOTS

Output 1: Hyperparameter Tuning & Optimization Summary

Displays configurable learning rate, iterations, test input, and resulting optimized parameters (w^* , b^*) alongside initial and final converged MSE costs.

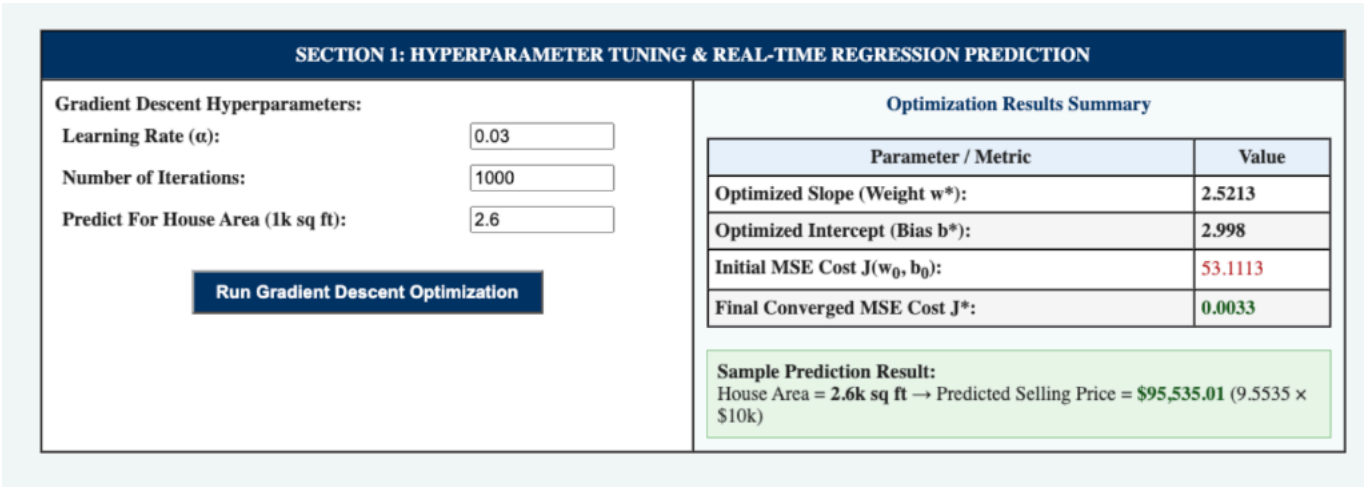


Figure 7.1: Section 1 — Hyperparameter tuning controls and optimization results summary.

Output 2: Gradient Descent Convergence Curve & Fitted Model

Visualizes the monotonic decay of MSE cost $J(w,b)$ across iterations alongside the fitted linear regression line over housing data points.

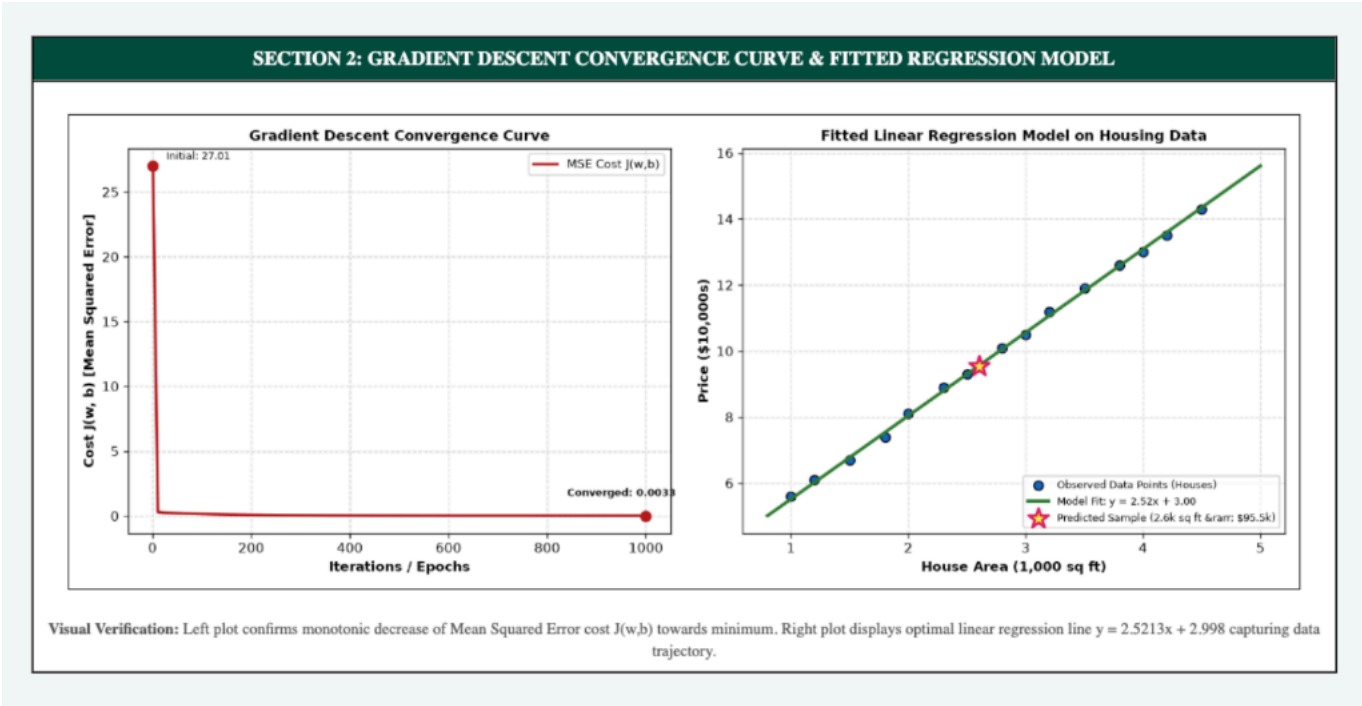


Figure 7.2: Section 2 — Gradient descent cost convergence curve and fitted linear regression line.

Output 3: Real Estate Dataset Samples & Residual Errors

Presents observed housing area values, true prices, model predictions, and absolute residual errors across samples.

SECTION 3: REAL ESTATE DATASET SAMPLES & RESIDUAL ERRORS				
House #	Area (1,000 sq ft) [x]	Actual Selling Price (\$10k) [y]	Model Prediction [$\hat{y}$]	Absolute Error $ y - \hat{y} $
1	1.0	5.6	5.52	0.08
2	1.2	6.1	6.02	0.08
3	1.5	6.7	6.78	0.08
4	1.8	7.4	7.54	0.14
5	2.0	8.1	8.04	0.06
6	2.3	8.9	8.8	0.10
7	2.5	9.3	9.3	0.00
8	2.8	10.1	10.06	0.04
9	3.0	10.5	10.56	0.06
10	3.2	11.2	11.07	0.13
11	3.5	11.9	11.82	0.08
12	3.8	12.6	12.58	0.02
13	4.0	13.0	13.08	0.08
14	4.2	13.5	13.59	0.09
15	4.5	14.3	14.34	0.04

Figure 7.3: Section 3 — Dataset sample evaluation and residual prediction error table.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSAxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_7/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_7/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_7/requirements.txt
- **Full Page Screenshot:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_7/screenshots/full_page.png

CONCLUSION

In this experiment, Linear Regression was implemented completely from scratch in Python without relying on pre-built machine learning libraries. Utilizing analytical gradient descent equations, optimal weights (w^*) and bias (b^*) were iteratively discovered for housing valuation data. The resulting Gradient Descent curve demonstrated stable monotonic cost convergence, and the fitted regression model reliably generalized predictions on unseen feature samples.